

AI/ML Intern Technical Assessment (Version B)

Worked answer key with explanations — Section A (MCQ) and Section B (Theory & Coding)

Note: this is study material to help you understand the reasoning behind each answer — work through the logic yourself before relying on it, since that's what will actually hold up in an interview.

Section A — Multiple Choice (15 marks)

Q1.

A DataFrame `orders` has columns `customer_id` and `order_value`. Which expression returns the average order value per customer, keeping only customers who placed more than five orders?

Correct answer: C

`orders.groupby('customer_id').filter(lambda g: len(g) > 5).groupby('customer_id')['order_value'].mean()` first keeps only groups (customers) with more than 5 rows, then computes the mean. Option B filters individual order VALUES greater than 5, not order COUNT, which is a different (wrong) condition. Option D checks whether the mean itself is above 5, not the count of orders. Option A never filters by count at all.

Q2.

Consider: `SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.id = o.customer_id WHERE o.status = 'shipped'`; Which statement is correct?

Correct answer: B

LEFT JOIN keeps all customers, including those with no orders (NULL order columns). But the WHERE clause runs after the join and checks `o.status = 'shipped'` — NULL never equals 'shipped', so every customer without a shipped order (including those with no orders at all) gets filtered out. The net effect behaves like an INNER JOIN restricted to shipped orders, even though it's written as a LEFT JOIN.

Q3.

A client's request times out and it's unclear whether the server processed it. Which statement about retrying is correct?

Correct answer: A

PUT is idempotent: sending the same PUT request multiple times leaves the resource in the same final state, so it's safe to retry. POST typically creates a new resource each time it's called, so retrying a timed-out POST risks creating a duplicate (e.g. double order, double payment).

Q4.

Which of the following is an example of data leakage?

Correct answer: D

A feature like `days_since_account_frozen` only exists (is non-null/meaningful) after a default has already occurred, meaning it encodes information from the future/outcome directly into the input features. Training on it means the model 'cheats' using information it would never have at real prediction time. The other options are standard, correct practices, not leakage.

Q5.

Validating a model on 3 years of loan applications that will score future applications. Which validation strategy is most appropriate?

Correct answer: B

TimeSeriesSplit trains on earlier data and validates on strictly later data, which mirrors how the model will actually be used in production (scoring applications that haven't happened yet). Random/shuffled splits (*KFold*, *StratifiedKFold*, *LeaveOneOut*) let future information leak into training, giving an overly optimistic and unrealistic performance estimate.

Q6.

A spam filter wrongly sending legitimate email to junk is far more costly than letting spam through. Which metric should be prioritised?

Correct answer: A

Precision on the spam class = $TP / (TP + FP)$, where a false positive here means a real, legitimate email was wrongly marked as spam. Prioritising precision directly minimises that costly mistake. Recall would instead focus on catching every spam email, which is the less costly error in this scenario.

Q7.

Training accuracy is 61% and validation accuracy is 60%, both flat for many epochs. Which action is most likely to help?

Correct answer: C

Both training and validation accuracy are low and close together — this is a classic underfitting signature (high bias), not overfitting. The fix is to increase model capacity or improve optimisation (better learning rate, longer training), not add more regularisation (A, which would make underfitting worse) or stop early (B, which addresses overfitting, not this problem).

Q8.

Data forms elongated, crescent-shaped clusters with scattered outliers between them. Which statement is true?

Correct answer: D

DBSCAN groups points by density and can find arbitrarily shaped clusters (like crescents) while explicitly labelling sparse/scattered points as noise rather than forcing them into a cluster. K-Means assumes roughly spherical, similarly-sized clusters based on distance to a centroid, so it performs poorly on elongated, non-convex shapes.

Q9.

A tensor of shape (32, 200, 6) is passed through `nn.LSTM(input_size=6, hidden_size=64, num_layers=2, batch_first=True)`. What is the shape of the first returned value (the output sequence)?

Correct answer: B

With `batch_first=True`, the LSTM's output sequence keeps the shape (batch, seq_len, hidden_size) — here (32, 200, 64). It returns one 64-dimensional hidden output per timestep, for every one of the 200 timesteps, for each of the 32 samples in the batch. (2, 32, 64) would instead be the shape of the final hidden state `h_n`, not the output sequence.'

Q10.

Training a binary classifier whose final layer produces a single raw value per sample. Which is correct?

Correct answer: A

`nn.BCEWithLogitsLoss` is designed to take the raw, un-activated logit directly and applies the sigmoid internally in a numerically stable way (using the log-sum-exp trick), avoiding precision issues that can occur if you apply sigmoid yourself first and then pass a saturated probability into a separate loss function.

Q11.

Why does doubling the input sequence length more than double the compute cost of a standard self-attention layer?

Correct answer: C

In self-attention, every token computes an attention score against every other token, producing an $n \times n$ attention matrix for a sequence of length n . Doubling n means the attention computation grows to $(2n)^2 = 4 \times$ the original cost — quadratic, not linear, scaling.

Q12.

Your RAG corpus was indexed six months ago with one embedding model. You upgrade the query-side code to a newer embedding model but do not re-index the documents. What happens?

Correct answer: D

Different embedding models place text into different, incompatible vector spaces. If queries are now embedded with a new model while the stored document vectors still reflect the old model, similarity search compares vectors that don't share a common geometry, so retrieval quality collapses. The whole corpus must be re-embedded with the new model to restore correct retrieval.

Q13.

Which scenario is the strongest case for fine-tuning rather than RAG?

Correct answer: B

Fine-tuning is well suited to teaching a model a consistent behaviour, format, or tone that should apply regardless of the topic (e.g. always writing in a rigid house report structure). RAG is better suited to injecting up-to-date or private factual knowledge (options A, C, D are all about accessing/citing specific, changing documents, which is RAG's core strength, not fine-tuning's).

Q14.

Your agent has a tool available, but the model never calls it and answers from its own knowledge instead. What is the most likely cause?

Correct answer: A

Tool-calling models decide whether and when to call a tool based largely on the tool's name and description. If the description doesn't clearly signal when the tool is relevant, the model may not recognise that the current request needs it, and will fall back on its own parametric knowledge instead.

Q15.

An HTTP Request node in your n8n workflow intermittently fails when the upstream API is briefly unavailable. What is the appropriate way to handle this?

Correct answer: C

n8n's built-in 'Retry On Fail' setting on a node lets you specify a retry count and wait interval to automatically recover from transient failures, while routing genuinely persistent failures to an error branch or a dedicated error workflow for visibility and handling — this is the standard, purpose-built mechanism for exactly this situation.

Section B — Theory & Coding (75 marks)

Q16. End-to-End Machine Learning Pipeline (15 marks)

(a) Split strategy + Pipeline with ColumnTransformer (6 marks)

Because the model will score **future** applications, a random split would let rows from later dates leak into training, letting the model implicitly learn from patterns that wouldn't exist yet in production. The data must be split **chronologically**: train on the earlier portion, test on the most recent portion, so the test set genuinely simulates unseen future data.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv("applications.csv", parse_dates=["application_date"])
df = df.sort_values("application_date")

# chronological split: last 20% by time held out as test
cutoff = df["application_date"].quantile(0.8)
train_df = df[df["application_date"] <= cutoff]
test_df = df[df["application_date"] > cutoff]

numeric_features = ["applicant_age", "annual_income",
                    "loan_amount", "employment_years"]
categorical_features = ["loan_purpose", "home_ownership", "region"]

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

clf = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(class_weight="balanced",
                                       random_state=42))
])

feature_cols = numeric_features + categorical_features
X_train, y_train = train_df[feature_cols], train_df["defaulted"].map({"Yes": 1, "No": 0})
X_test, y_test = test_df[feature_cols], test_df["defaulted"].map({"Yes": 1, "No": 0})

clf.fit(X_train, y_train)
```

(b) Two problems with the colleague's approach (4 marks)

Split problem: `train_test_split` with a random shuffle mixes rows from all three years into both train and test, so later applications leak into training and earlier ones leak into the test set. This lets the model implicitly see patterns from 'the future' relative to some test rows, inflating reported performance beyond what it would achieve on genuinely new applications.

Metric problem: only 12% of applications defaulted, so the classes are heavily imbalanced. A model that simply predicts 'No' (no default) for every applicant would already score around 88% accuracy without learning anything useful. 94% accuracy on this dataset is barely better than that naive baseline and says very little about whether the model actually identifies defaulters — precision/recall/F1/AUC on the minority class are needed instead.

(c) Cost-asymmetric metric + tuning code (5 marks)

Approving a future defaulter costs about 13× more than wrongly rejecting a good applicant (£80,000 vs £6,000), so false negatives (missed defaulters) are far more expensive than false positives. This means **recall on the defaulted class** should be weighted more heavily than precision — a practical way to encode that is an F-beta score with $\beta > 1$ (e.g. F2), which weights recall roughly 2–4× more than precision, approximating the cost ratio.

```
from sklearn.metrics import make_scorer, fbeta_score
from sklearn.model_selection import GridSearchCV

# beta > 1 weights recall (catching defaulters) more than precision
f2_scorer = make_scorer(fbeta_score, beta=2, pos_label=1)

param_grid = {
    "classifier__n_estimators": [100, 300],
    "classifier__max_depth": [5, 10, None],
}

search = GridSearchCV(clf, param_grid, scoring=f2_scorer, cv=5)
search.fit(X_train, y_train)
print(search.best_params_, search.best_score_)
```

Q17. Deep Learning with PyTorch (15 marks)

(a) Model class for (32, 200, 6) → 4 fault classes (6 marks)

```
import torch
import torch.nn as nn

class FaultLSTM(nn.Module):
    def __init__(self, input_size=6, hidden_size=64, num_layers=2, num_classes=4):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # x: (batch=32, seq_len=200, channels=6)
        out, (h_n, c_n) = self.lstm(x)
        # out: (32, 200, 64) -- one hidden output per timestep
        last_step = out[:, -1, :] # (32, 64)
        logits = self.fc(last_step) # (32, 4)
        return logits
```

I feed the **last timestep's** output (`out[:, -1, :]`) into the dense layer, because the LSTM processes the sequence in order, so by the final timestep its hidden state has already incorporated information from all 200 earlier timesteps. It acts as a learned summary of the whole vibration sequence, which is exactly what a sequence-classification head needs (equivalently, this is the same value as `h_n[-1]`, the last layer's final hidden state).

(b) Training loop for one epoch (6 marks)

```
model = FaultLSTM()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def train_one_epoch(model, train_loader, val_loader):
```

```

model.train()
total_loss = 0.0
for X_batch, y_batch in train_loader:
    optimizer.zero_grad()
    outputs = model(X_batch)
    loss = criterion(outputs, y_batch)
    loss.backward()
    optimizer.step()
    total_loss += loss.item()

model.eval()
val_loss, correct, total = 0.0, 0, 0
with torch.no_grad():
    for X_val, y_val in val_loader:
        outputs = model(X_val)
        loss = criterion(outputs, y_val)
        val_loss += loss.item()
        preds = outputs.argmax(dim=1)
        correct += (preds == y_val).sum().item()
        total += y_val.size(0)

return total_loss / len(train_loader), val_loss / len(val_loader), correct / total

```

optimizer.zero_grad() clears gradients left over from the previous batch — PyTorch accumulates gradients by default, so without this they'd add up incorrectly across batches. **loss.backward()** runs backpropagation, computing the gradient of the loss with respect to every model parameter. **optimizer.step()** then updates the weights using those gradients according to the optimizer's update rule (e.g. Adam).

model.eval() switches layers that behave differently at train vs. inference time (dropout, batch norm) into inference mode — e.g. dropout stops randomly zeroing activations. It does *not* stop gradient tracking on its own. **torch.no_grad()** separately disables gradient computation entirely, saving memory and compute during validation/inference since no backward pass is needed. Both are required together: eval() for correct layer behaviour, no_grad() for efficiency.

(c) 71% accuracy, one class is 71% of data (3 marks)

The model has collapsed into always predicting the majority class — a trivial shortcut that gives the illusion of learning ('majority-class collapse'). Loss still decreased during training because cross-entropy loss averaged over a heavily imbalanced dataset genuinely gets lower as the model becomes more confident in predicting the dominant class for nearly every sample, even though it has learned nothing about distinguishing the other three fault types.

Two fixes: (1) use a class-weighted loss (nn.CrossEntropyLoss(weight=...)) or oversample the minority classes (e.g. a WeightedRandomSampler) so the model is penalised more for missing rare classes. (2) Stop evaluating with raw accuracy alone — monitor macro-F1 or a per-class confusion matrix during training so this kind of collapse is caught immediately instead of looking like 'good' accuracy.

Q18. Generative AI — RAG with Access Control (15 marks)

(a) Architecture: ingestion + query time (5 marks)

Ingestion: (1) extract text from each client's source documents; (2) split into overlapping chunks; (3) generate embeddings for each chunk; (4) store each vector in the vector database along with metadata that includes **client_id**, document id, source, and page number.

Query time: (1) authenticate the user and resolve their **client_id** server-side from their session/token (never trust a client-supplied value); (2) take the new question plus recent conversation turns and pass them through a query-rewriting step so a follow-up like 'what about for contractors?' becomes a standalone question (e.g. 'What is the notice period for contractors?'); (3) embed the rewritten standalone question; (4) query the vector database with a **hard metadata filter** on **client_id**, so only that tenant's vectors are ever searched, not just ranked lower; (5) retrieve the top-k chunks; (6) build the final prompt from the system prompt, retrieved context, and the conversation; (7) call the LLM and return the answer, ideally with source citations.

Enforcing isolation: the **client_id** filter must be applied at the vector-database query itself (a hard pre-filter), derived only from the authenticated session, never from anything the client sends in the request body — optionally, using separate indexes/namespaces per client adds a second layer of isolation.

Follow-ups: store recent turns as conversation memory, and run every new user message through an LLM-based rewriting step that resolves references to earlier turns into a self-contained query before it's embedded and retrieved.

(b) Minimal implementation (6 marks)

```
import uuid

def chunk_text(text, chunk_size=500, overlap=50):
    chunks, start = [], 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size - overlap
    return chunks

def ingest_document(text, client_id, doc_id, embed_fn, vector_store):
    for i, chunk in enumerate(chunk_text(text)):
        vector = embed_fn(chunk)
        vector_store.upsert(
            id=str(uuid.uuid4()),
            vector=vector,
            metadata={"client_id": client_id, "doc_id": doc_id,
                    "chunk_index": i, "text": chunk},
        )

def rewrite_query(chat_history, new_question, llm_call):
    prompt = (f"Chat history:\n{chat_history}\n\n"
              f"Rewrite the latest question '{new_question}' as a standalone "
              f"question, resolving references to earlier turns.")
    return llm_call(prompt)

def retrieve(query, client_id, embed_fn, vector_store, top_k=5):
    query_vector = embed_fn(query)
    return vector_store.query(
        vector=query_vector,
        top_k=top_k,
        filter={"client_id": client_id},    # hard scope, enforced server-side
    )

SYSTEM_PROMPT = (
    "You are an internal assistant. Answer ONLY using the provided context "
```

```

        "chunks. If the answer is not in the context, say you don't have that "
        "information. Never reference documents outside the provided context."
    )

def build_prompt(question, retrieved_chunks):
    context = "\n\n".join(c["metadata"]["text"] for c in retrieved_chunks)
    return f"{SYSTEM_PROMPT}\n\nContext:\n{context}\n\nQuestion: {question}"

```

(c) Three retrieval-stage failure causes + fine-tuning case (4 marks)

- 1. Follow-up question embedded literally** (e.g. 'what about for contractors?' retrieved on its own) pulls irrelevant chunks. Fix: the query-rewriting step from part (a), resolving it into a standalone question before embedding.
- 2. Chunking splits a table or answer across a boundary**, so the relevant fact is fragmented across two chunks and never fully retrieved together. Fix: use structure-aware or larger overlapping chunks, or a parent-document retrieval strategy that expands a matched small chunk back out to its full surrounding section.
- 3. Embedding/index mismatch** — the corpus was indexed with an older embedding model and never re-embedded after an upgrade (as in Q12), so query and stored vectors no longer share the same space. Fix: re-index the full corpus whenever the embedding model changes, and track an index-version tag.

Fine-tuning over RAG: when every response must follow a rigid, consistent structure and house tone regardless of topic (as in Q13) — that's a behavioural/style requirement, not a need for fresh factual retrieval, which is what fine-tuning is naturally suited for.

Q19. Model Deployment & AI/ML Engineering (15 marks)

(a) FastAPI batch prediction service (6 marks)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import List
import joblib
import pandas as pd

app = FastAPI()
MODEL = None

@app.on_event("startup")
def load_model():
    global MODEL
    MODEL = joblib.load("model.pkl") # load once at startup, not per request

class Application(BaseModel):
    applicant_age: int
    annual_income: float
    loan_amount: float
    employment_years: float
    loan_purpose: str
    home_ownership: str
    region: str

class PredictRequest(BaseModel):
    applications: List[Application] = Field(..., max_items=500)

class Prediction(BaseModel):
    prediction: str
    probability: float

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict", response_model=List[Prediction])
def predict(request: PredictRequest):
    if MODEL is None:
        raise HTTPException(status_code=503, detail="Model not loaded")
    try:
        df = pd.DataFrame([a.dict() for a in request.applications])
        probs = MODEL.predict_proba(df)[: , 1]
        preds = MODEL.predict(df)
        return [
            {"prediction": "Yes" if p == 1 else "No", "probability": float(pr)}
            for p, pr in zip(preds, probs)
        ]
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

(b) Dockerfile (4 marks)

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd -m appuser
USER appuser

EXPOSE 8000
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# run: docker run -p 8000:8000 loan-model-api
```

python:3.11-slim is a minimal base image, keeping the image small and reducing the attack surface compared to a full OS image. **requirements.txt is copied and installed before the rest of the app code** so Docker's layer cache is reused — when only application code changes (not dependencies), Docker skips reinstalling packages, making rebuilds much faster. Running as a **non-root user** limits the damage if the container is ever compromised, since the process won't have root privileges inside (or, via privilege-escalation bugs, outside) the container — a standard security best practice many platforms now require.

(c) More rejections after v3 deploy, no errors thrown (5 marks)

Investigation: compare the distribution of predictions (and predicted probabilities) that v3 produces on a recent batch against what v2 would have produced on the exact same batch (a shadow/replay comparison), and separately compare the raw input feature distributions being sent now vs. before deployment. Also check whether any feature's encoding or category set changed silently between training and this deployment (e.g. a categorical value the OneHotEncoder didn't see during training).

What had to be instrumented in advance: logging of which model version served each prediction, the input features for each request, the output prediction/probability, and business KPIs (approval rate) over time — without these logs there's no way to reconstruct what actually changed after the fact.

Rollback: keep the previous model (v2) versioned in a model registry or accessible via a config/environment flag, so serving can be pointed back to it immediately without a code redeploy — ideally through a canary or blue-green deployment so the switch is instant and low-risk.

Training–serving skew is when the features or preprocessing seen at serving time differ from what the model was trained on (e.g. a subtly different imputation or encoding path in production vs. training), so the model behaves unexpectedly even though it hasn't changed — it's a pipeline consistency bug. **Data drift**, by contrast, is when the real-world input distribution itself genuinely shifts over time (e.g. applicant demographics change), even though the pipeline stayed perfectly consistent — the model is working exactly as trained, but the world it's scoring has moved.

Q20. Automation & Agentic AI with n8n (15 marks)

(a) Workflow design: nodes in order (6 marks)

1. **Email Trigger** (IMAP/Gmail) — fires on a new email in the shared mailbox; outputs email metadata plus the PDF attachment as binary data.
2. **Extraction node** (AI/LLM node, with OCR fallback) — takes the PDF and extracts supplier name, invoice number, line items, and total into structured JSON.
3. **HTTP Request node** — calls the internal purchase-order REST API using the extracted supplier/invoice number, returning the matching PO's total and line items as JSON.
4. **IF/Switch node** — compares invoice total vs PO total: passes if a PO was found AND the difference is within 2%, otherwise routes to the mismatch branch.
- 5a. **(Match branch)** HTTP Request node posts the invoice to the accounting system, followed by an Email/Gmail node sending the automated acknowledgement to the supplier.
- 5b. **(Mismatch branch)** Slack node posts the invoice and PO details to the finance-approver channel; no acknowledgement email is sent on this path.

Data flow: raw email + PDF binary → structured invoice JSON → invoice JSON merged with PO lookup response → boolean match decision → either (accounting payload + acknowledgement) or (Slack approval payload).

(b) Tool JSON schema + agent reasoning loop (5 marks)

```
{
  "name": "lookup_purchase_order",
  "description": "Look up a purchase order by supplier name (and PO number if known) to retrieve its expected total",
  "parameters": {
    "type": "object",
    "properties": {
      "supplier_name": {
        "type": "string",
        "description": "Name of the supplier as it appears on the invoice"
      },
      "po_number": {
        "type": "string",
        "description": "Purchase order number if known, otherwise omit"
      }
    },
    "required": ["supplier_name"]
  }
}
```

Given the extracted invoice fields and a system prompt that says a PO lookup tool is available for verification, the model decides — based on the task of checking the invoice against a PO — that it needs the tool, and emits a tool call with the `supplier_name` (and `po_number` if it has one). n8n actually executes the HTTP request and returns the PO's JSON result back to the model as a tool result. The model reads that result, compares it against the invoice total it already extracted, and once it has enough information to produce a final match/mismatch decision it stops calling tools and returns its final structured output; if the first lookup didn't return a usable result, it could call the tool again with different parameters before finishing.

(c) Production edge cases (4 marks)

- (i) **Malformed JSON from the LLM:** wrap the parsing step in a Code node with try/catch; on failure, either re-prompt the LLM with the parse error asking it to correct its own output (a self-repair retry), or route the item straight to the Slack approval branch rather than letting the workflow crash.
- (ii) **Duplicate invoice emailed 3 days apart:** before posting to accounting, check an idempotency store (a database lookup or a Set/dedupe node) keyed on `invoice_number` + `supplier_name`; if a matching record already exists, skip re-posting and stop that branch rather than risk double payment.
- (iii) **Scanned image with no text layer:** a plain text-extraction step will return empty or garbage text for a pure image.

image PDF, so route it through an OCR step (or a vision-capable LLM) first before attempting field extraction.

Study tip: for Section B, practice re-deriving the code from the bullet points above without looking, out loud, explaining each design choice — interviewers usually probe the 'why' behind a line of code more than the syntax itself.